Handles: results (D-168 to D-187, updated 2026-10-25)

Companion to the research note (in the PDF it follows this section). All runs use the same model (Qwen3-VL-4B + D207 adapter), the same real-time runtime, the same 12 tasks and development seeds 1060-1062. Each variant was flown 36 times. These are development results; the held-out run comes at the end.

The handles built so far

Handle	What it is	What it changes
H1 typed task contract	The planner writes a small typed program (visit, pass, condition, follow, goto, search) instead of plain text steps	What the planner hands over
H2 kind-aware executive	One owner runs that program: asks the camera at a branch, flies through openings, follows for the stated time, advances only on evidence	Who runs the plan, and how
H5 measured openings	Gaps in walls measured by the drone's range sensor, given to checks and fly-through steps; questions the sensor can answer are routed to it	How perception reaches the plan
H7 look while following (negative)	A follow step looks again while moving instead of stopping each time	Timing of perception during tracking

None of these adds a box to the architecture diagram. They change the logic between the boxes.

Step 1: H1 and H2, alone and together (FULL stack as base)

Variant	Passes / 36	Planning tasks / 12
FULL (no handles)	21	0
+ H1 only	24	0
+ H2 only	21	0
+ H1 and H2	27	3 (ordered visit 3/3)

Neither handle helps planning alone. A typed plan that nobody executes is inert, and an executive with nothing typed to execute does nothing. Together they solve the first planning task. This matches the scaffold literature: interaction between components matters more than any single module.

How the version-1 handles failed is evidence too. The planner wrote a correct if-then program, and a strict validator rejected it for one extra field. Fixing the connecting logic, not the model, is what moved the score:

Version	H1 + H2
v1	18/36
v2	26/36
v3	27/36

Step 2: adding H5 (measured openings) on top of H1 + H2

Variant	Passes / 36	Planning / 12	What changed
H1 + H2	27	3	-
+ H5 (facts given to the model)	28	4	the gate is reached every time, but the right branch only 1/3
+ H5 with check routing	30	6 (gate 3/3, ordered visit 3/3)	sensor-answerable questions answered by the sensor

The small model ignored the correct measured facts in its answer: it said "no left opening" when the sensor measured one. Routing the question to the sensor fixed it on every seed. This is another connecting-logic decision, not a better model.

Step 3: what did not work (rounds 3 to 5)

Try	Aimed at	Result
H7 look while following	follow target	The drone flew half as far: looks took GPU time from the pilot
Wall-end detour (fly past the measured wall end)	closing passage	Flew beside the wall end, not past it; cost the gate one seed
Detour behind the wall + progress along the planned path	closing passage	Correct aim, but the drone stops at the wall corner; cost the gate one seed

All three are kept behind switches, off in the final stack, as measured negatives.

Why these two tasks stay open: they are time-critical. On follow, the first move comes after about 5.7 s of model time (planning 2.6 s plus the first look 3.1 s), and by then the marker is already out of reach; even the privileged C0 fails it. On closing passage, the wall closes at 3 s, while the planner is still writing the plan. Changing the logic between the boxes cannot remove model latency.

Step 4: remove-one ablation of the final stack

Arm	Passes / 36	Planning / 12	Tasks lost
H1 + H2 + H5 (final)	30	6	-
minus H5	27	3	if-then gate
minus H2	21	0	gate, ordered visit, approach and hover
minus H1	21	0	gate, ordered visit, obstacle slalom
no handles (FULL)	21	0	-

H1 and H2 are each necessary: take either one out and the stack falls back to the no-handle score. H5 only helps on top of both.

Caveat: every task gave the same result on all three seeds, so each arm behaves more like 12 outcomes than 36 trials. The differences are whole tasks that flip, not noise, but the held-out seeds must confirm them.

Step 5: the same handles on the other planner architectures

The handles act on the planner -> executor connection, so they fit unchanged on the three architectures that have one. Nothing is added to any architecture. C1-C6 have no such connection, so these handles do not apply

there.

Architecture	Without handles	With H1 + H2 + H5	Gain
FULL (classical pilot)	21	30	+9
C13 (event planner, VLA pilot)	22	27	+5
C10 (timed planner, VLA pilot)	16	21	+5

- C13 + handles solves the if-then gate 3/3, like FULL + handles.
- C10 + handles picks the right opening, then its timed planner review overrides the executive and sends the drone off course. Two owners of the plan conflict, which is the reason H2 exists.
- Ordered visit fails with the VLA pilot. The order is right, but the pilot is slow, so the first pillar is reached at 48 s and time runs out before the second.

Step 6: handles for the architectures without a planner (C2-C8)

H1, H2 and H5 need a planner. C2-C8 have other connections, and three handles act on those, again without adding any box:

Handle	Connection	Fits
H3 search policy	target not visible -> bounded camera search, when the mission says find	C2-C6
H4 evidence-based completion	the stop decision: stated standoff / hold, object passed below	C2-C8
H9 measured distance	the VLA picks the target box, the depth sensor gives the distance	C7, C8

The code audit found that H3 and H4 already existed, but only on the planned path, so FULL had them and C2-C5 never did. These were hidden handles inside the "full" architecture.

Architecture	C2	C3	C4	C5	C6	C7	C8
without handles	12	15	15	18	18	0	0
with handles	19	21	23	23	23	6	6

- Approach and hover: 0/3 -> 3/3 on C2, C3, C4 and C6 (H4 reads "hover 1 m, hold 2 s").
- Turn and search: 0/3 -> 3/3 on C2-C5 (H3).
- C7 / C8, the VLA alone: 0 -> 6. The VLA boxes the target correctly; the sensor fixes its distance. They still fail the obstacle tasks, because they fly without the path planner.
- Planning tasks stay 0/12 on all of them. With no planner, nothing holds a branch or an order.

Step 7: which part limits each architecture (perfect-part swaps)

One part at a time is replaced by a perfect version, delivered after the model's own measured time, with everything else unchanged. Passes / 36, development seeds.

	as is	perfect plan (after 5.5 s)	perfect grounding (after 3 s)	perfect pilot
FULL + H	30	30	30	-
C13 + H	27	-	27	-
C3 + H	21	-	24 (overturned car)	-
C0 (true goal)	28	-	-	27 with the VLA pilot

- The pilot is never the limit, and perception only on the upside-down car.
- With handles, the plan is no longer the limit either.
- What is left is structure (no planner: no planning tasks), the 4B model's perception of the car, and timing (follow; closing passage, where the wall closes before any plan exists).
- Lesson for the method: a perfect part handed over instantly also removes the model's delay. That fooled me twice, so every perfect part is now delivered after the model's own time.

Step 8: a second round of handles for every architecture, then plateau

Architecture	round 1	round 2	added
FULL + H	30	30	plan check (H11), no gain
C13 + H	27	26	plan check, no gain
C10 + H	21	25	H8: the timed review waits while the step makes progress
C2 + H	19	24	H12: the VLM's coordinate format is accepted (it was fixed only inside C3's verifier)
C1	9	9	H3 + H4, no gain

Hidden handles: FULL's planned path already carried H3 / H4, and C3's verifier carried the format repair that C2 lacked. Some "architecture" differences are really these.

Where this stands

Configuration	Passes / 36	Planning / 12
Privileged C0 (true goals)	28	4
Best vanilla architecture (C13)	22	0
FULL	21	0
FULL + H1 + H2 + H5	30	6
C13 + H1 + H2 + H5	27	3

Every architecture improves with the handles that fit it (C2 12 -> 19 ... C7 0 -> 6). Only the planner stacks gain planning tasks.

Same model, same tasks, same seeds. The difference is only the handles. The handled stack even beats the privileged C0 on the planning tasks, because C0 has the true goal but no plan logic.

Task maps: what happens when

Each page is one task (seed 1060). Every panel is one architecture, seen from above. The line is the drone's path, coloured by time (dark = start, yellow = 60 s). Grey blocks are walls, red blocks are walls that appear during the flight, and triangles are objects (black edge = the target). Rows: each group of architectures, with its handled version (+H = the handles that fit that architecture) in the row directly below. All 12 tasks for all three seeds are in reports/paper_maps/.

Architectures and handles in the task maps

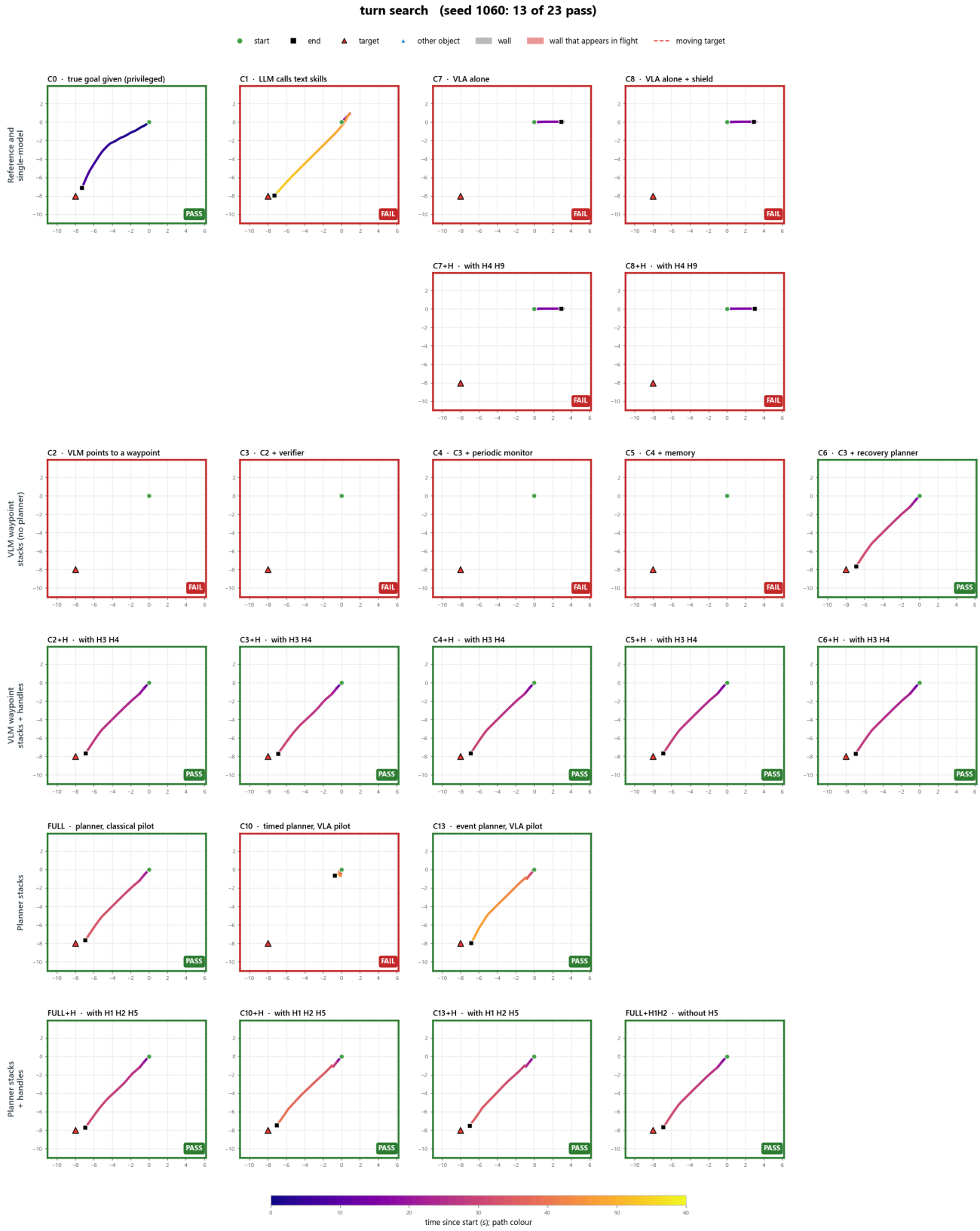
C0	Oracle: the true goal is given to a classical pilot. Privileged ceiling.
C1	Skill agent: a text-only LLM calls goto / approach / detect / scan / hover.
C2	VLM points at the target in the image; the point becomes a waypoint.
C3	C2 plus a verifier that checks each waypoint before it is flown.
C4	C3 plus a monitor that reviews progress on a timer.
C5	C4 plus memory of past goals and outcomes.
C6	C3 plus a recovery planner called on events (lost target, stall).
C7	VLA alone: the trained VLA writes its own goal line and flies it.
C8	C7 plus the safety shield and envelope verifier.
FULL	Planner + VLM grounding + verifier + monitor + event recovery + memory; classical pilot.
C10	As FULL, the planner runs on a timer; the VLA is the pilot.
C13	As FULL, the planner runs on events; the VLA is the pilot.
H1	Handle: typed task contract (visit / pass / condition / follow / goto / search).
H2	Handle: one executive runs the contract by step kind.
H5	Handle: measured openings from the range sensor; sensor answers what it can.
H3	Handle: a find / search mission starts the bounded camera search itself.
H4	Handle: completion on evidence: stated standoff / hold, object passed below.

Same model (Qwen3-VL-4B + D207 VLA adapter), runtime, path planner, shield and tasks in every panel. '+H' = the handles that fit that architecture.

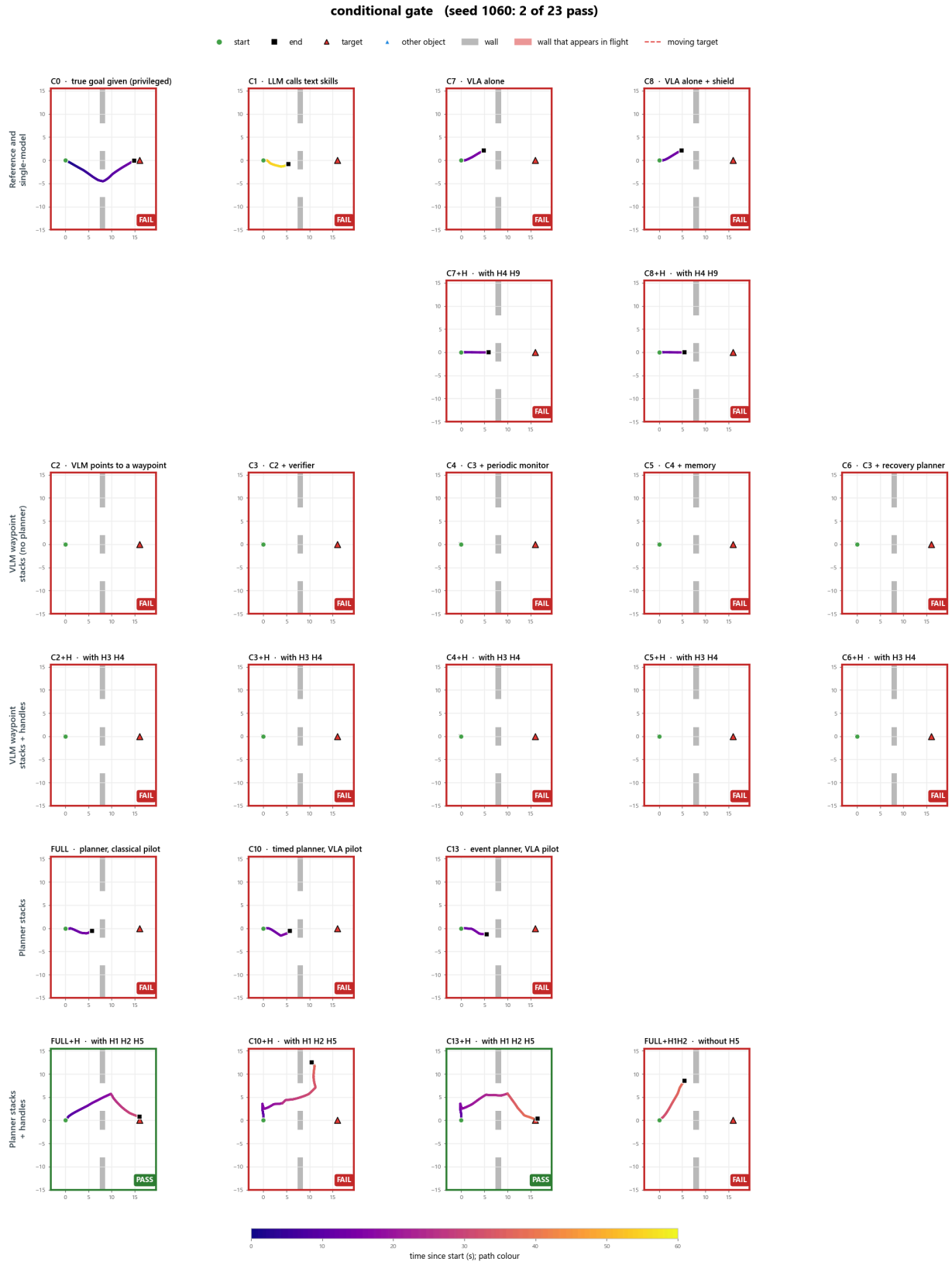
Legend. What each architecture code means.



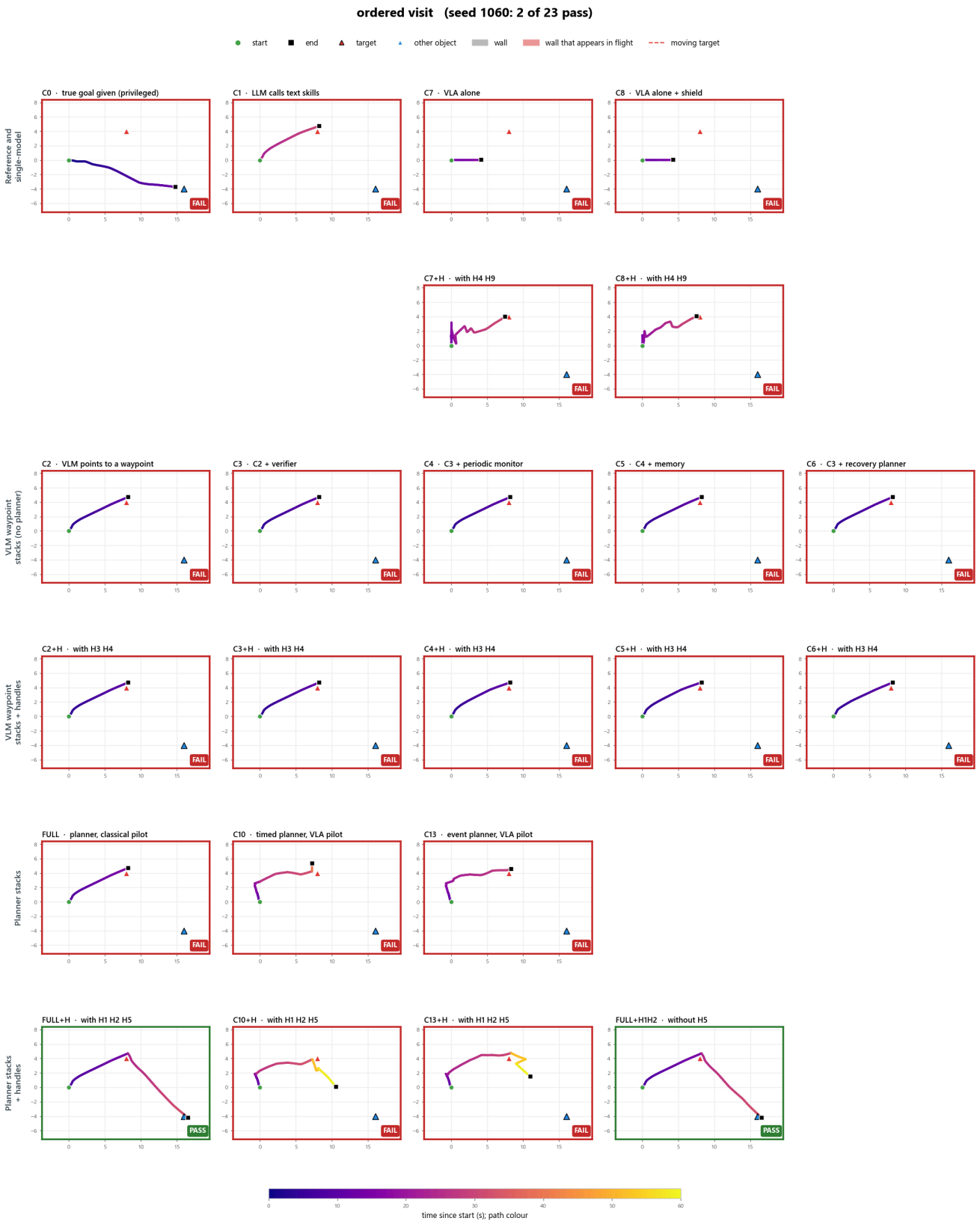
Approach and hover. Every VLM waypoint stack gets to the pillar, but C2, C3, C4 and C6 stop at the wrong distance and never hold. With H4 they read "about 1 m, hold for 2 s" from the mission and pass, as FULL does through its planner.



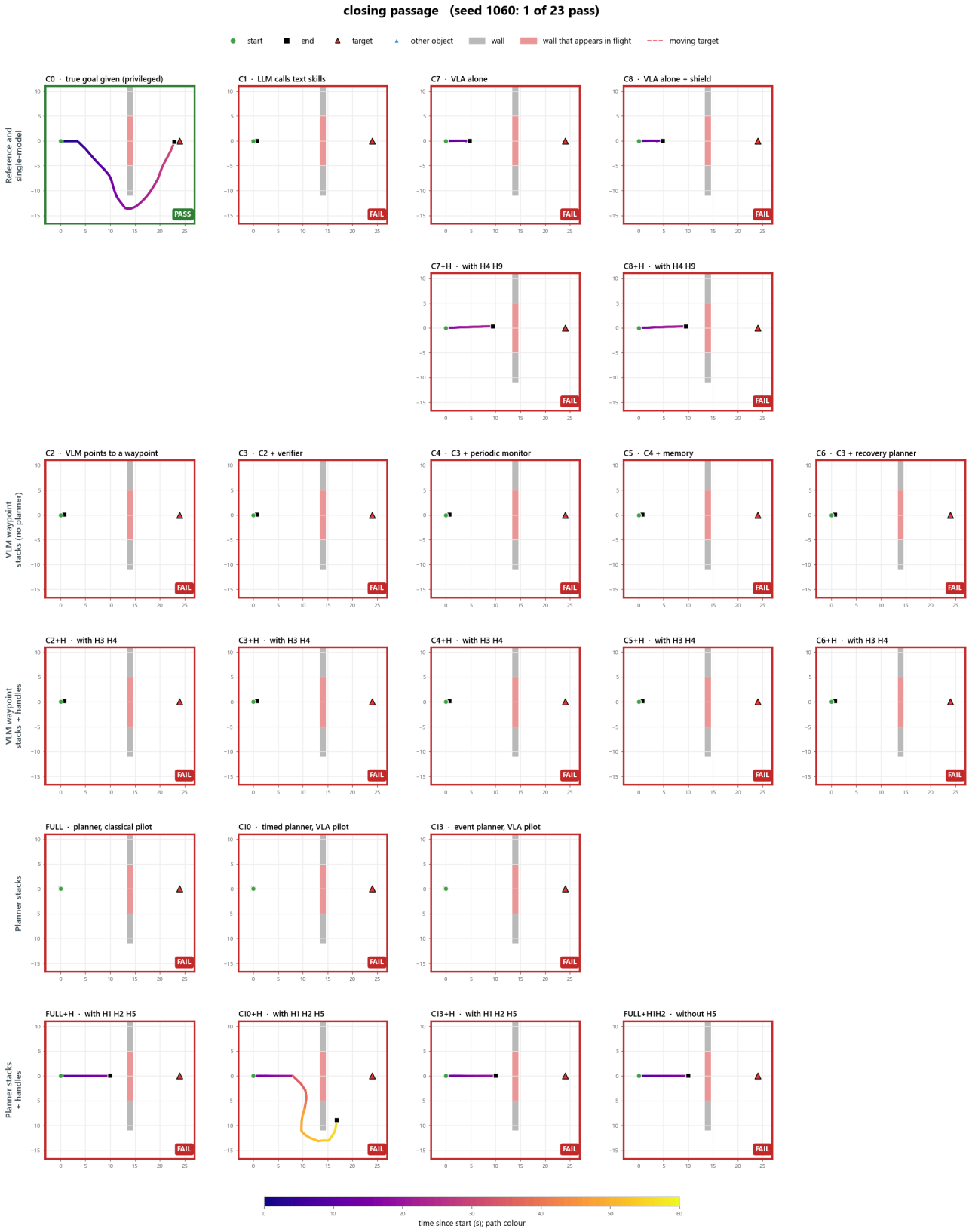
Turn and search. The pillar starts behind the drone. C2-C5 never turn, because nothing authorizes a search without a planner. With H3 they turn, find it and fly to it.



If-then gate. Only FULL + H and C13 + H pass: the range sensor answers which opening is clear, and the drone flies through it and on to the pillar. Every vanilla stack stops before the wall. C10 + H takes the correct opening, then its timed planner sends it north.



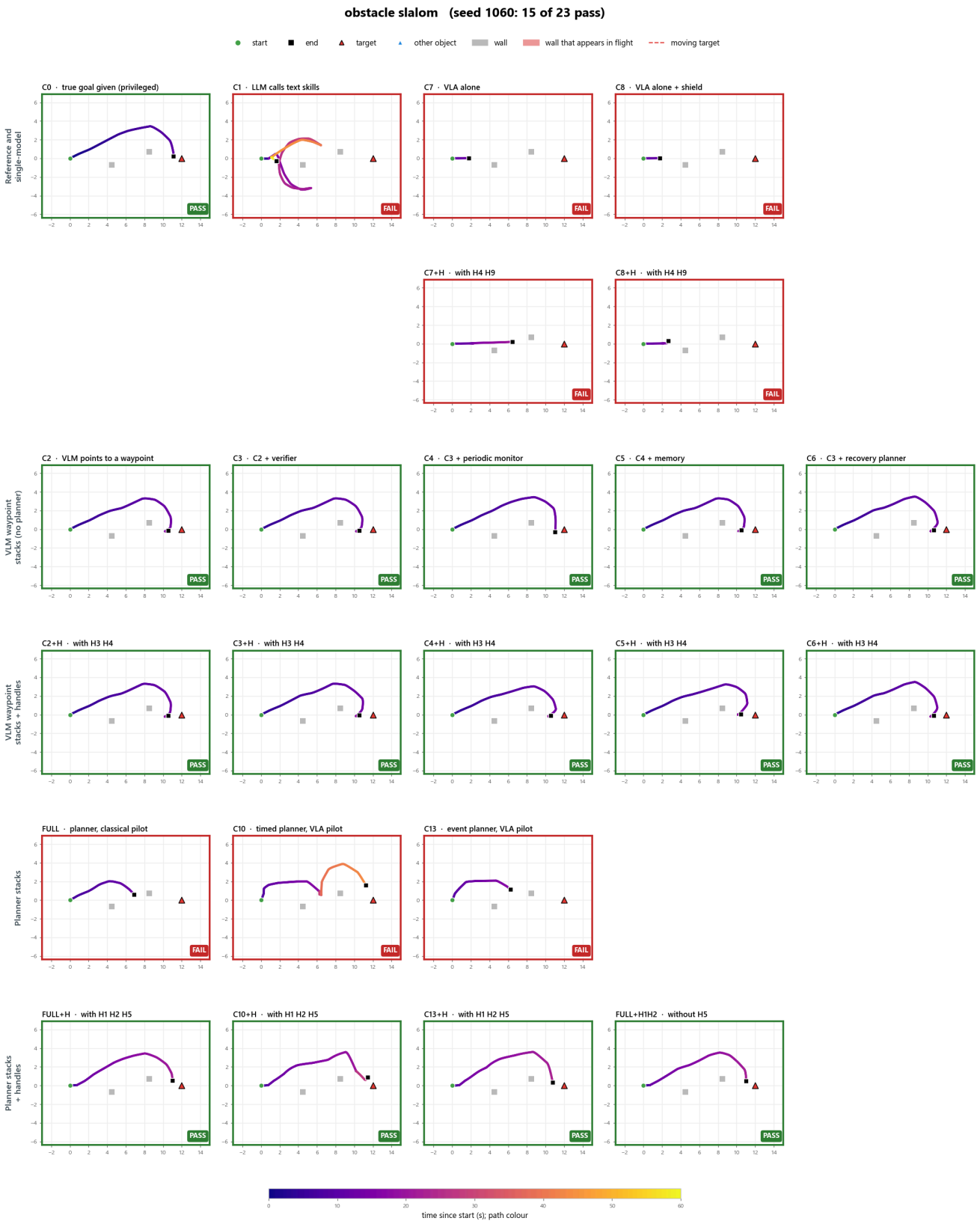
Ordered visit. Every vanilla stack flies to the red pillar and stops: the two-step mission becomes one visit. With the typed plan and executive (FULL + H, FULL + H1H2) the drone visits red, then flies on to blue. C10 + H and C13 + H keep the order, but the slow VLA pilot is still near red when time runs out (yellow).



Closing passage. Only C0, which is given the true goal, flies around the wall that closes at 3 s. The handled stacks fly up to the wall and stop; C10 + H goes around the end too late. This task stays open: the wall closes while the planner is still thinking.



Follow target. No architecture passes. The vanilla stacks chase the marker down the track but stay too far behind it. The handled stacks keep the stated 4-6 m band only after a late start, and end at 20 s as the mission says. The first move comes about 5.7 s in, which is too late.



Obstacle slalom. The vanilla planner stacks (FULL, C10, C13) stop between the posts; with the handles, the same stacks reach the target like the simpler C2-C6 stacks.

What makes an LLM/VLM drone architecture work: the "handles" (D-167, 2026-10-22)

NO CODE CHANGED. Research note. PDFs: `papers/handles/` (README there).

Question

Our paper matrix (D-166) runs the textbook architectures (C0-C13) on one model and one runtime. Seed 1060: every model-driven architecture solves the basic tasks but none of the four planning tasks (conditional gate, ordered visit, closing passage, follow).

The user's observation: the systems that work (drone_control, AerialClaw) do so because of logic that lives outside the architecture diagram: - the prompt logic; - the task contracts; - the state hand-offs; - the flow between parts.

The question: what does the field say these "handles" are, and how much do they matter?

Vocabulary

The field calls this the **scaffold** or **harness** around the model: the interfaces, state tracking, verification and feedback loop. AgentSpec 2026 uses "scaffold"; the UAV MCP DroneServer 2026 uses "agentic harness". LLM-Modulo calls the checking part "external critics / verifiers". Inner Monologue calls the feedback part "closed-loop language feedback" / "success detection".

Evidence from the literature

Work	Finding relevant to the thesis	Link
LLM-Modulo (Kambhampati et al., ICML 2024)	Autoregressive LLMs cannot plan or self-verify by themselves; they work as approximate knowledge sources inside a loop with external model-based critics/verifiers	2402.01817
AgentSpec (Chen et al., 2026)	"Agent performance is governed by scaffold compatibility and interaction effects rather than isolated module strength"; "composition can outweigh model scale"; memory helps only when its representation matches the reasoner. DeliveryBench, same model: ReAct 18.57 -> 34.70 with a memory bank	2606.14674
Embodied Agent Interface (Li et al., NeurIPS 2024)	Breaks LLM embodied decision-making into goal interpretation, subgoal decomposition, action sequencing and transition modelling. Most failures are module-level (hallucinated objects, unmet preconditions, missing / misordered steps), invisible in success rate alone	2410.07166
Inner Monologue (Huang et al., CoRL 2022)	Feeding success detection and scene description back into the planner prompt is what lets an LLM planner retry, replan and finish long instructions; open-loop plans could not	2207.05608
ProgPrompt (Singh et al., 2022)	Program-like prompts: action APIs, example programs, natural-language comments and precondition assertions with recovery. Comments and assertions measurably raise success / executability	2209.11302
SayCan (Ahn et al., 2022)	The LLM's choice is multiplied by a learned affordance (can this skill succeed here?); the language model alone proposes infeasible steps	2204.01691
Code as Policies (Liang et al., 2022)	Success depends on the API (perception and control primitives) and the few-shot examples handed to the model	2209.07753
TypeFly (Chen et al., 2023)	Drone LLM planning through a small DSL (MiniSpec) plus a "query" skill and replanning; ablations show these design choices, not the LLM alone, decide complex-task success and halve cost / latency	2312.14950
CoPAL (2023); Garrabé et al. (2024)	Mid-execution failures need multi-level corrective feedback. Missing / mischaracterised subgoals are a main failure mode; an "expected outcomes" module plus feedback recovers them, especially with small local models	2310.07263, 2411.05474
Bode et al. (2024)	Prompt-engineering technique and combination change service-robot task completion; prompts released	2410.22997
DroneServer / MCP harness (Ramos Silva & Burke, 2026)	98 typed drone tools, server-side mission state, tiered validation, independent geofence: 8 of 11 LLMs completed 90-100 % of the simulated missions; no geofence escape in 110 trials. Reliability across models comes from the harness (no harness-off ablation reported)	2601.15486
AerialClaw (Li et al., 2026)	A brain-skill-runtime loop: SOUL / BODY documents (identity and capability bounds), soft-skill strategy notes, a validated one-skill-per-turn JSON decision, explicit done / stuck verdicts, execution feedback, memory, reflection	2606.12142

Work	Finding relevant to the thesis	Link
Our earlier 94-paper UAV survey (drone_control, `survey_llm_planning_uavs_systematic`, Table 8)	Typed tools help, "but every tool call needs preconditions, state grounding, and safety authorization". The main gap is semantic-to-metric grounding. Benchmarks rarely compare architectures under identical constraints	local

What the field tried and dropped: the "LLM writes the whole plan once" (open-loop) design. Inner Monologue and CoPAL replaced it with closed loops. SayCan and LLM-Modulo replaced "trust the LLM's step" with a feasibility or critic check. Free-form text plans were replaced by typed APIs or DSLs (ProgPrompt, Code as Policies, TypeFly).

Evidence from our own repositories (checked against the code)

drone_control (`src/agent/`). The working loop is a set of handles around the models:

Handle	Where	What it does
Pinned reasoning order + task-policy library	`task_policies.py`	The VLM prompt fixes the order *mission -> required evidence -> current evidence -> gap -> action*; one task policy is picked once per mission. Written after run 20260812_134500, where all 14 VLM reasons cited detections and none cited coverage
Capability allow-list	`capability_registry.py`	A deterministic allow-list given to the LLM mission compiler
Typed mission contract	`mission_compiler.py` (PlannerAdapter), `contracts.py`	The LLM response is converted into a typed contract; invalid messages raise
Proposal gate	`proposal_gate.py`	First deterministic gate on model proposals (mission / subgoal context)
Single sequencing owner	`mission_executive.py`	"Single owner of reactive mission sequencing"
Fused, freshness-limited world snapshot	`blackboard.py`	Async perception fused into one timestamped snapshot with freshness limits; only a "mature" track counts as found
Decision FSM	`decision_manager.py`, `skill_state_machine.py`	World state -> pilot skill
Progress monitor	`progress_monitor.py`	Low-rate, OnFly-inspired

Its recent fixes are handle fixes, not model fixes: - S10: a routine VLM answer no longer releases a candidate mid-inspection. - B12: the close-range check turns thinking off. - S9: the crowd gate uses the standard LOF threshold.

AerialClaw in our testbed (C1, D-44 / D-105). It is the one paper family that passed its gate (5/5 grid, 5/5 object search). The handles it had: - SOUL/BODY documents; - soft-skill strategy; - runtime precondition checks; - explicit done / stuck verdicts; - per-turn feedback; - perception rendered as labelled metric detections.

D-85 already noted that the labelled detections gave C1 a perception input the VLM families did not have.

The live shared-Qwen stack, same weights, no training.

Period	Result	What changed (handles only)
D-143 -> D-145	0/4 -> 4/4 numeric tasks	monitor status semantics, numeric completion contract, ordered-step preservation
D-150 -> D-158	0/9 -> 5/9 capability tasks	world-pose contract, standoff fix, altitude hold, object memory, standoff / hold read from step text, monitor "hold" with fresh near depth counts as stop

The D-166 matrix. The vanilla C configurations, which have only the diagram's modules, pass basic tasks and fail every planning task. The ordering among them (C5/C6 above C2-C4) comes from the components that act as handles: memory and recovery.

Taxonomy for the paper (handles as experimental factors)

ID	Handle	Literature	Our evidence	Target tasks
H1	Typed task contract with branch / sequence semantics (conditions, ordered steps, completion evidence per step)	ProgPrompt, TypeFly, EAI subgoal errors	D-144/145 ordered steps; conditional_gate "branch flattened"	conditional_gate, ordered_visit
H2	Single sequencing owner / executive with explicit step state	drone_control MissionExecutive, AerialClaw runtime	D-145, D-158	ordered_visit, closing_passage
H3	Pinned reasoning order + task-policy library in prompts	drone_control task_policies, Bode 2024	run 20260812	search / perception tasks
H4	Evidence-based completion (success detection)	Inner Monologue, CoPAL	D-147/148/158 stop rules	approach_hover, overturned_vehicle
H5	Perception -> semantic state hand-off (labelled, metric, fresh)	Code as Policies API, Blackboard	C1 labelled detections (D-85), D-147 grounding	all visual
H6	Feasibility / precondition critics before execution	SayCan, LLM-Modulo, DroneServer	verifier C3, SUPER (D-165)	obstacles, gate
H7	Feedback of outcomes into the next prompt + bounded memory	Inner Monologue, AgentSpec memory	C5 / C6, skill history in C1	closing_passage, follow
H8	Recovery invocation policy (when to call the reasoner)	PMR, OnFly	C6 vs C10 vs C13	closing_passage

Implication and proposed next experiment (not started)

The paper's first finding: **vanilla architectures do not work by themselves; the handles do.** To measure this rather than assert it: 1. Keep the D-166 vanilla matrix as the "diagram only" condition. 2. Build the handles H1-H8 as switchable components on the same runtime (several already exist in the live stack). 3. Run add-one (vanilla + Hk) and remove-one (full - Hk) ablations on the same tasks and seeds.

The prediction from the literature and our history: the handles explain more variance than the choice among C configurations.

Out of scope

- Training a bigger model or new VLA weights: the thesis is about the scaffold at fixed weights.
- Cloud models: one local model is a paper rule (D-159).
- Task-specific scripted behaviours that read evaluator truth: never allowed (AGENTS.md rules). Handles must use public mission text and sensed state only.